RESEARCH PROJECT | PROGRESS REPORT

# LLM Character Operations

Connecting language understanding with exact character and string tools.

System overview and work completed

Report to Professor | 14 October 2026

# Why we are building this system

Goal: let the model understand the task, then use exact code for the character-level work.

## Language model

Interprets the wording, identifies the required operation and extracts its inputs.

→

## Deterministic tools

Carry out counting, lookup and text editing using fixed, repeatable rules.

**Example**

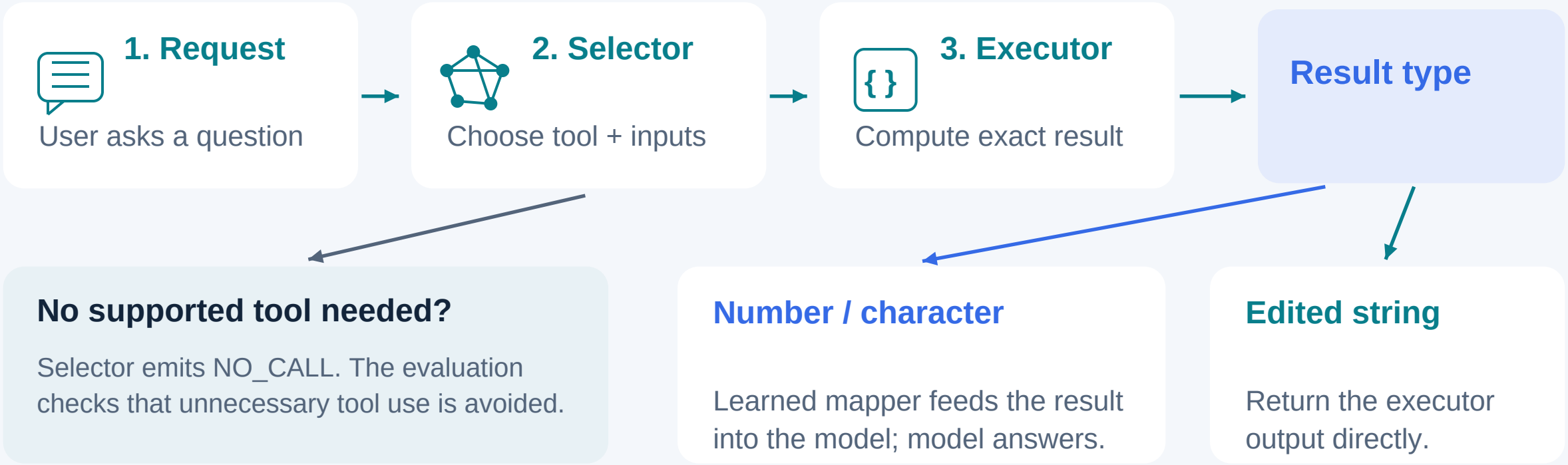
How many "a" characters are in "banana"?

COUNT_CHAR → **3**

Research question: can this combination improve reliability and handle unfamiliar wording?

# How a request moves through the system

The model selects an operation; the executor runs the predicted call; the output follows its result type.




An exact executor still needs the correct call. Wrong tool choices or inputs can produce wrong answers.

# What the system learns

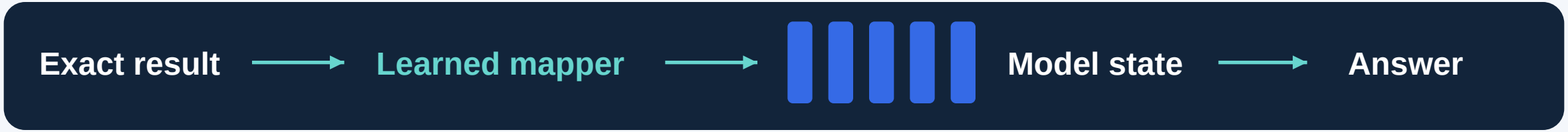
Two separate learning stages around a Qwen3-8B backbone; Qwen3-4B is available for replication.

### Stage 1: choose the tool

A small trainable adapter learns the operation, exact inputs and when to make no call.

### Stage 2: use the result

A small mapper learns how to feed a number or single character into the model's internal state.




The mapper handles seven number/character operations. Its insertion layer is chosen using development data.

# What we have prepared so far

A repeatable data and experiment pipeline covers 12 operations, including examples that require no tool.

### Count and measure

Character count, string length, word count and length of a selected word.

### Find and retrieve

Find a character position; retrieve a character from text or a selected word.

### Edit and reverse

Insert text or words; reverse a string or a selected word.

**Original dataset**

**50,000 examples**

**Expanded dataset**

**150,000 examples**

Includes longer sentence contexts; kept separate from the original protocol.

Both releases: 70% train / 10% dev / 10% test / 10% new-wording test; 37% no-call controls.

# Current progress and next steps

The implementation and experiment framework are in place; full research conclusions require completed runs.

## Implemented

12 deterministic tools and dataset generation.

Selector and mapper training, full pipeline evaluation, recovery and reporting.

## Prepared for validation

Separate 150k / two-GPU training protocol.

Matched comparisons: unaided model, direct-answer training, ordinary tool feedback and mapper pipeline.

**Train** → **Choose on dev** → **Test + new wording** → **Compare**

Next: verify server runs, repeat training across seeds, and compare accuracy, speed and GPU memory.